

NT-Transformer: A Non-Pretrained Encrypted Network Traffic Classification Model

Zhenzhen Yan¹, Member, IEEE, Lizhi Peng², Member, IEEE, Peiqiang Liu, Yingshuo Bao³,
and Bo Yang⁴, Member, IEEE

Abstract—Network traffic classification plays an indispensable role in network management, Quality of Service (QoS), and cybersecurity. With the widespread encryption techniques applied to network traffic, it has become increasingly challenging to classify network traffic into different management groups accurately. In recent years, pre-training Transformer-based models have been successfully applied to Natural Language Processing (NLP), and researchers have also introduced such models into encrypted network traffic analysis. However, besides the similarities of words in NLP and byte codes in network traffic, there exist essential differences between them, which may cause inefficacy of the pretrained model when being applied to new traffic data. In this paper, we propose a non-pretrained encrypted network traffic classification model based on Transformer called NT-Transformer, which can directly learn labeled network traffic features at two levels of granularity, namely, byte level (uni-gram or bi-gram) and flow level (packet size and packet inter-arrival time), without the relatively expensive pre-training procedure of unlabeled data. This method is validated on three public datasets and three sets of recently collected network traffic data. Experimental results indicate that in some scenarios, pretrained models offer limited performance gains when applied to new encrypted network traffic data not encountered during pre-training, and NT-Transformer with uni-gram byte representation outperforms the state-of-the-art models in terms of pushing the F1 score up by 0.25% - 2.24%.

Index Terms—Encrypted network traffic classification, transformers, byte representation, uni-gram, pre-training, deep learning.

I. INTRODUCTION

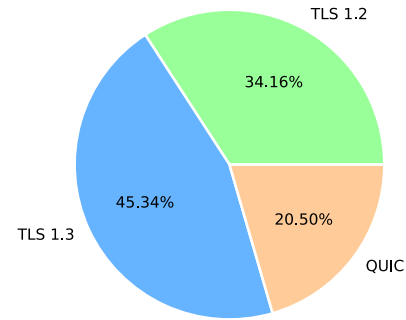
NETWORK traffic classification, aiming at identifying and classifying network traffic into different management groups, plays an essential role in network planning, network monitoring, Quality of Service (QoS), and intrusion/malware detection. In recent years, with the increasing requirement

Received 1 July 2025; revised 1 December 2025 and 14 January 2026; accepted 8 April 2026. Date of publication 13 April 2026; date of current version 28 April 2026. This work is supported by Shandong Provincial Natural Science Foundation under Grant No. ZR2022QF151, in part by the the Opening Fund of Shandong Key Laboratory of Ubiquitous Intelligent Computing, in part by the Jinan Scientific Research Leader Studio, China, under Grant No.202228114, and in part by the TaiShan Scholars Program under Grant No. tstp20240828. The associate editor coordinating the review of this article and approving it for publication was N. Limam. (Corresponding author: Zhenzhen Yan.)

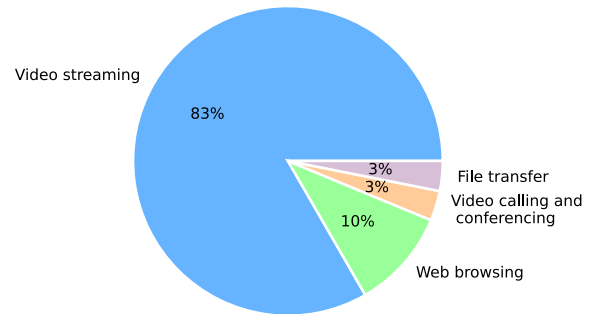
Zhenzhen Yan and Peiqiang Liu are with Shandong Technology and Business University, Yantai 264005, China (e-mail: zhengina27@163.com).

Lizhi Peng and Bo Yang are with Shandong Key Laboratory of Ubiquitous Intelligent Computing, University of Jinan, Jinan 250022, China, and also with the Quan Cheng Laboratory, Jinan 250103, China.

Yingshuo Bao is with Beijing Normal University, Zhuhai 519087, China. Digital Object Identifier 10.1109/TNSM.2026.3683410



(a) Top Three Encrypted Traffic Protocols



(b) Top Four Encrypted Traffic Classes

Fig. 1. Global encrypted traffic protocols and classes.

and demand for data security and user privacy, network traffic encryption has been widely utilized. According to the Google Transparency Report, the percentage of data that is encrypted across Google products and services has grown from 50% to 96% [1] in the last ten years. Fig. 1 illustrates the contributions to the global encrypted traffic by the top three encryption protocols and the top four traffic classes. TLS 1.3 and QUIC(which also incorporates elements of the TLS 1.3 standard) account for 65.84% of encrypted traffic [2], [3], and video streaming is responsible for over 80% of encrypted traffic volume [4]. The ubiquity of encryption has protected Internet users and data to a great extent, while at the same time posing major challenges to network traffic classification. Cybercriminals and malware can take advantage of advanced encryption techniques, such as TLS, VPN, etc., to evade the detection of firewalls, Intrusion Prevention Systems (IPS), and

Intrusion Detection Systems (IDS), which makes the work of accurate network traffic classification indispensable.

There are three main categories of methods for network traffic classification: port-based, Deep Packet Inspection (DPI)-based, and Machine Learning (ML)-based [5]. The accuracy of port-based methods is relatively low due to the commonly used port disguises and dynamic ports. The DPI-based methods focus on inspecting packet payload to find keywords or patterns, which can raise privacy concerns because they need to dig deep into the content of packets. Besides the privacy issues, DPI-based methods often incur high computational overhead and are only applicable for unencrypted network traffic [6]. The ML-based methods can be further divided into two classes of models [7] based on whether the input to ML models is raw traffic or not: 1) divide-and-conquer model, where packet and flow features are manually selected or designed from raw network traffic as the input to ML models. The performance in terms of accuracy and computational cost is highly dependent on the quality and quantity of the hand-selected or hand-designed features; 2) end-to-end model, where raw traffic is fed directly into ML models without any human expertise, which may be prone to misinterpretation and overgeneralization. Either one of these two models may overlook some useful information that the other model covers, e.g., the divide-and-conquer model ignores the byte-level details, and the end-to-end model doesn't take the flow-level knowledge into consideration.

In recent years, Transformer-based deep learning frameworks have achieved great breakthroughs in Natural Language Processing (NLP) [8], [9], [10], Computer Vision (CV) [11], speech processing [12], multi-modal [13], and many other fields [14]. These deep learning frameworks usually consist of two steps: pre-training and fine-tuning. During pre-training, the models learn from a tremendous amount of unlabeled data, which is typically computationally expensive, but compromising on data can lead to performance degradation. For fine-tuning, the models' parameters can be initialized with the pretrained values as a starting point, and further tuned using labeled data from specific downstream tasks.

The Transformer-based pre-training and fine-tuning models have been applied to encrypted network traffic classification in the last two years [15], [16], [17], [18], where the unlabeled traffic payload data are transformed to pattern-preserved tokens for pre-training, and a small portion of these unlabeled data are used in the fine-tuning phase after adding the labels back. These models have demonstrated superior performance compared to the classic deep learning models, such as Convolutional Neural Network (CNN) and Recurrent Neural Network (RNN), with the overhead of a massive amount of data and computing power required in the pre-training step. The pretrained models are built with the intention of "once and for many", which means that once a pretrained model is created, it is expected to facilitate many future downstream tasks. However, there exist significant differences between words/sentences/paragraphs in a natural language and bytes/flows/sessions in network traffic. During the pre-training step of NLP, phrases, grammar rules, idiomatic expressions, etc., of a language are learned. These learned

language features and rules usually do not change rapidly over time, which is the key point to the success of pretrained models in the NLP field because we use languages to communicate between individuals, interpret each other's ideas, and make ourselves understood, and fast-evolving rules contradict these purposes of languages. Another aspect of natural languages is that encryption is rarely used, because it contradicts the purpose of communication. Such characteristics of languages make the pretrained models advantageous when dealing with various downstream tasks, in that there is no need to build models from scratch, and optimization can usually be reached with a relatively insignificant amount of fine-tuning data and computational cost.

The story is different when analyzing network traffic. Besides the application (e.g., Gmail, Youtube, etc.) and service (e.g., email, video, etc.) fingerprints applied to the data, the content being transferred is largely dominated by end users, e.g., in a chat scenario, the end users may send any combinations of words, pictures, videos, and voice messages, without following any specific rules about which item should come after which. For security purposes, encryption has been widely applied to network traffic. Languages aim to convey meaning, while network traffic tries to prevent understanding of its content until it is decrypted. These facts indicate that there are more unpredictable features to learn in the network traffic classification scenario compared to NLP, and pretrained models using one pool of datasets may not add the expected value when applied to another pool of previously unseen datasets. The instability characteristic of network traffic undermines the reusability and transferability of pretrained models, which usually require a significant amount of time and computational resources to be pretrained.

In this paper, we propose a non-pretrained encrypted network traffic classification model, called **Network Traffic-Transformer (NT-Transformer)**, which can directly learn labeled network traffic features from both byte level (uni-gram or bi-gram) and flow level (packet size and inter-packet arrival time), without the relatively expensive pre-training procedure of unlabeled data. The main contributions are as follows:

- We validate that when data used for fine-tuning is sampled from the same pool as data used for pre-training, pretrained models can both improve the success rate of fine-tuning tasks and expedite the tuning process; However, pretrained models often provide limited gains when applied to new, previously unseen datasets.
- We propose a hybrid model, namely NT-Transformer, combining the divide-and-conquer and end-to-end model, in which both packet and flow level features are encoded as the input to the model. The byte-level features enable the model to learn contextual semantics, while the flow-level features provide information about the fingerprints of applications and services, as well as user behaviors, network conditions, etc.
- NT-Transformer is able to classify network traffic at three levels of granularities, i.e., application level, flow level, and content level, and NT-Transformer with uni-gram encoding plus flow level encoding outperforms the state-of-the-art models on one public dataset and three newly

collected datasets by 0.25% - 2.24%, which demonstrates its effectiveness and generalization ability.

The rest of the paper is organized as follows. Section II summarizes the related work. Section III describes our proposed model, NT-Transformer, in detail. Experiments and analyses are presented in Section IV. Finally, we conclude this paper in Section V.

II. RELATED WORK

In this section, we briefly summarize the methods that have been applied to the network traffic classification field, including fingerprint-based methods, feature engineering-based methods, deep learning methods, and Transformer models. A comparative overview of these related works is provided in Table I, which details their methodological categories, classifiers, utilized features, use of multi-granular features, adoption of two-phase training, and evaluation datasets.

A. Fingerprint-Based Methods

Unlike traditional DPI-based methods that can raise privacy concerns and fail when being applied to encrypted traffic, FlowPrint [19] introduces a semi-supervised approach for constructing mobile app fingerprints by combining temporal, device, destination, and packet size features, without inspecting packet payload information. However, these fingerprints have the potential to be tampered under VPN or proxy scenarios, making traffic in these virtual networks evade detection.

B. Feature Engineering-Based Methods

Feature engineering-based methods (or divide-and-conquer methods [7]) divide the complex network traffic classification problem into several simpler sub-problems. The workflow usually includes three sequential steps: 1) Task-specific features are first hand-designed; 2) These features are then extracted or generated from raw network traffic; 3) Finally, the feature data is input to classic statistical-based or behavioral-based machine learning approaches [44]. Based on the traffic discriminators summarized by Moore et al., there can be as many as 249 features to characterize Internet flows [45], and feature engineering-based methods generally adopt a subset of these for their specific research goals, e.g., AppScanner [20] generates statistical features from flows, BIND [21] adopts both burst size and burst time as input to ML classifiers, and Xie et al. propose a new feature: byte code distribution [22], which is generated from raw traffic bytes. However, these methods heavily rely on human-engineered task-specific features, which may limit their generalizability.

C. Deep Learning Methods

Deep learning (DL), the hottest branch of ML that uses deep neural networks as learning methods, has been widely applied to network traffic classification with raw traffic as input to automatically learn discriminative features. In 2017, Wang et al. firstly tried to use Convolutional Neural Networks (CNNs) to learn features from raw traffic [7], [44]. Since then, many researchers have followed this way of applying DL

models. Sirinam et al. apply CNNs to extract representations from raw packet direction sequences [27]. Shapira and Shavitt [23], [24] convert packet sizes in consecutive time slices of flows into FlowPic images which are fed into a CNN. Based on this approach, Horowicz et al. [25] propose mini-FlowPic with augmentation techniques in few-shot learning. Liu et al. adopt Recurrent Neural Networks (RNNs) [28] to learn representative features from raw flows. Combination models of both CNNs and RNNs have been designed to obtain both packet and flow features [5], [33], [35], [36], [37] or both spatial and temporal characteristics [32], [46] [30]. Graph Neural Networks (GNNs) models [47] are also developed using raw traffic and packet/flow features [38], [39]. These DL-based methods have achieved superior performance until the attempt to apply Transformer-based models to the network traffic classification field is made.

D. Transformer Models

In recent years, Transformer-based models have pushed the performance on tasks in many fields to unprecedented levels, especially the advancements in NLP, CV, speech processing, etc. [48]. Since natural languages and communication protocols share some common characteristics, researchers have been inspired to apply Transformer models to the network traffic classification field. ConViTML [42] applies CNNs to the byte images created using the first 784 bytes of packets, and then divides the images into patches, which are then flattened as the input to Transformer. YaTC [18] processes each flow into a multi-level flow representation matrix. These works embed byte/packet/flow level information into fixed-sized images, but have not considered that some pixels may not enhance the traffic classification results, e.g., the encrypted packet payload pixels may not be as informative as packet header pixels. PERT [40], ET-BERT [15], and TrafficFormer [17] use byte-pair to encode the hexadecimal raw byte sequences, because with byte-pair, the range of byte values is comparable to the size of a common NLP vocabulary. However, these works mainly focus on how to prepare the network traffic to be as close as possible to NLP words in order to apply the Transformer model, and overlook the unique and intrinsic characteristics of network traffic. PEAN [41] and FlowBERT [16] use both raw bytes and length sequence as their inputs, which extends the type of tokens. They lack the analysis of whether their byte representations are optimal, and the packet inter-arrival time is also neglected.

To the best of our knowledge, existing Transformer-based traffic analysis models follow the framework of pre-training and fine-tuning, in which a model is pretrained on enormous unlabeled data to learn the contextual representations of network traffic, especially through the “mask” process that randomly masks a certain percent, e.g., 15%, of the tokens, to predict the masked token by fusing bidirectional (i.e., both the left and right) context [9]. When fine-tuning task-specific models, the downstream tasks can take advantage of the pretrained model to reduce the demand for labeled input data and computing power. However, besides the characteristics in common, one major difference between natural languages and network traffic is whether the representations learned can

TABLE I
RELATED WORKS IN NETWORK TRAFFIC CLASSIFICATION TASKS

Methods	Related Work	Classifier	Traffic Features	Features at Different Levels of Granularity	Two-phase Training	Datasets
Fingerprint-based methods	[19]	Clustering + Random Forest	destination IP, port number, timestamp, packet size and direction	✗	✗	ReCon, Cross Platform, Andrubis, Browser ^a
Feature engineering-based methods	[20]	Reinforcement Learning	54 statistical features from each flow	✗	✗	Dataset 1-5 ^a
	[21]	SVM + k-NN	burst size and burst time	✗	✗	HTTPS, TOR, App Dataset ^a
	[22]	C4.5 Decision Trees, SVM and BPNN	byte code	✗	✗	video traffic data ^a
Deep learning methods	[23], [24]	2D-CNN	packet size and inter-arrival time	✗	✗	ISCX, TAU ^a
	[25]	2D-CNN	packet size and inter-arrival time	✗	✗	UC-Davis QUIC, ISCX
	[7]	1D-CNN	first 784 bytes of packets	✗	✗	ISCX
	[26]	1D-CNNs	handshake headers	✗	✗	ISCX, UC-Davis QUIC datasets(a)
	[27]	CNNs	packet direction sequences	✗	✗	datasets(a)
	[28]	BiGRU	raw flow sequences	✗	✗	dataset ^a [29]
	[30]	2D-CNN + LSTM	source port, destination port, packet size, inter-arrival time, packet direction, and TCP window size	✗	✗	dataset ^a
	[31]	1D-CNN + LSTM	payload bytes of the initial packet and packet length sequences	✓	✗	dataset ^a
	[5]	CNNs + LSTM	first 784 bytes of L3 and three consecutive packets of each flow	✓	✗	ISCX
	[32]	1D-CNN + BiLSTM	first N packets of each flow, the first M bytes of each packet	✓	✗	ISCX, USTC-TFC2016
	[33]	1D-CNN + BiGRU	first 576/784 bytes of L4 payload, L4 payload length, TCP window size, packet inter-arrival time, and packet direction	✓	✗	MIRAGE-2019 [34]
	[35], [36]	1D-CNN + BiGRU	first 576/784 bytes of L4 payload, L4 payload length, TCP window size, packet inter-arrival time, and packet direction	✓	✗	ISCX
	[37]	1D-CNN + BiGRU	first 576 bytes of L4 payload, L4 payload length, TCP window size, packet inter-arrival time, packet direction, and Context input	✓	✗	MIRAGE-COVID-CCMA-2022 ^a
	[38]	GNNs	packet direction, packet length, packet burst, and packet ordering	✗	✗	dataset ^a
	[39]	GNNs	raw bytes, packet inter-arrival time, and meta features of flows	✓	✗	ISCX
Transformer models	[40]	Transformer	raw traffic	✗	✓	ISCX, dataset ^a
	[41]	Transformer	raw traffic	✗	✓	dataset ^a
	[15]	Transformer	first 64 bytes of L3	✗	✓	Cross-Platform, USTC-TFC, ISCX, CSTNET-TLS1.3
	[17]	Transformer	first 64 bytes of L3	✗	✓	ISCX, Browser, Cross-Platform, CICAndMal2017, CSTNET-TLS1.3, USTC-TFC
	[18]	Transformer	up to first 320 bytes of L3	✗	✓	ISCX, USTC-TFC, CICIOT2022, Cross-Platform
	[42]	CNNs + Transformer	first 784 bytes of packets	✗	✗	USTC-TFC, CICIOT2022
	[43]	CNNs + Transformer	first 784 bytes/128 bytes of L4 payload	✗	✗	TON_IoT, ISCX, USTC-TFC
	This paper	Transformer	first 40/60/80 bytes of L2 payload, packet size, packet inter-arrival time	✓	✗	ISCX, CSTNET-TLS1.3, USTC-TFC, datasets ^a

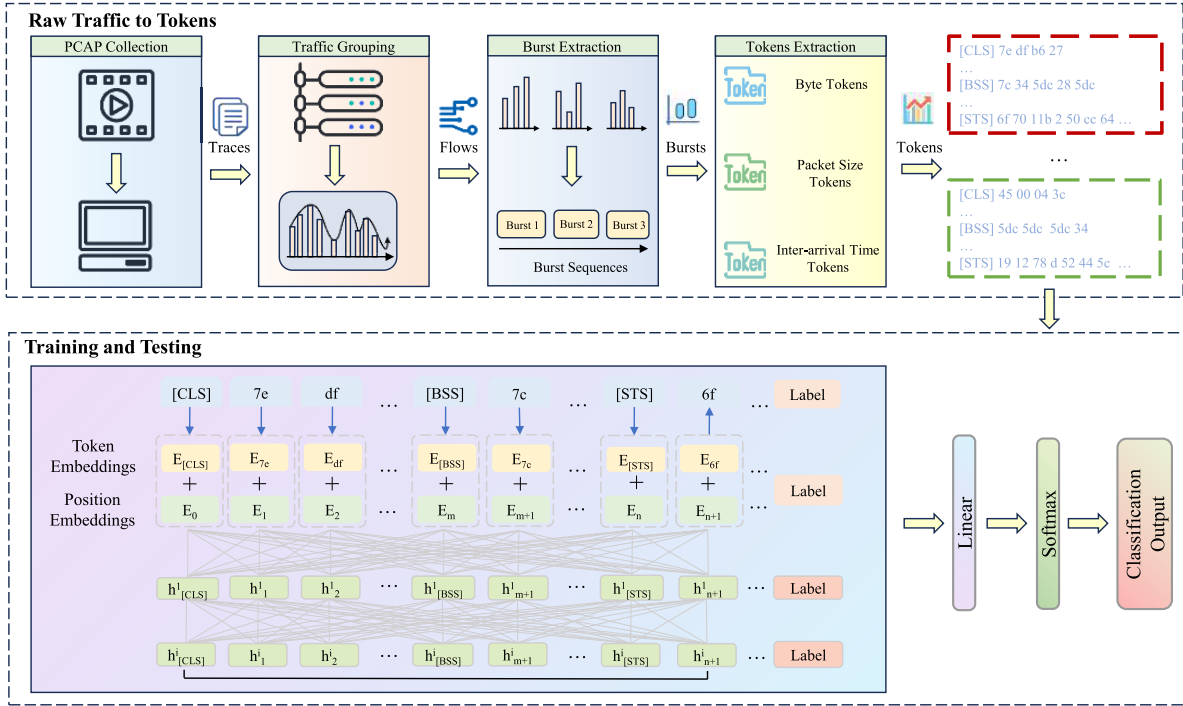


Fig. 2. NT-transformer framework.

represent a considerable amount of future downstream tasks, which can highly influence the reusability and effectiveness of the pretrained models.

III. NT-TRANSFORMER

NT-Transformer differs from the related Transformer-based models in that it learns from labeled input directly, without the pre-training step. Our hypothesis is: network traffic, in terms of packets and payload bytes, has the representations of actual data, user behaviors, application/service signatures, and network conditions. This combination of representations can be learned to classify network traffic within the lifespan of flows/sessions and a longer period if these representations stay relatively unchanged. Compared with natural languages, which have grammar rules, idiomatic expressions that are supposed to be invariable at least for years, network traffic can vary frequently when end users decide to send/receive something different, change their sending/receiving behaviors, or there is an application update, etc. The variability characteristic of network traffic can compromise the transferability and generalizability of pretrained models. An experimental analysis is provided in Section IV to discuss the reusability of pretrained models on previously unseen datasets.

The framework of NT-Transformer is shown in Fig. 2. We transform a combination of raw traffic bytes, packet size, and packet inter-arrival time into tokens as the input to the NT-Transformer model. NT-Transformer adopts a base Transformer encoder architecture comprising $L = 12$ layers (i.e., Transformer blocks), each with $A = 12$ self-attention heads. The model uses a hidden size $H = 768$ (i.e., dimension) and accepts input sequences of up to $N = 512$ tokens. This parameterization aligns with benchmark models including

PERT [40], ET-BERT [15], and TrafficFormer [17], ensuring equitable comparisons. This base configuration substantially lowers computational demands, often reducing training time by more than two-thirds on identical hardware compared to larger variants [49]. Nonetheless, pre-training with this base architecture remains resource-intensive, a finding consistent with existing literature [40] and corroborated in Section IV-E.

A. Raw Traffic to Tokens

Raw network traffic data in .pcap or .pcapng format is first cleaned and processed to remove noise and anomalies, e.g., ARP and DHCP packets are deleted; Ethernet header is removed, since it does not pertain to the end-to-end traffic classification task; IP addresses and port numbers are masked to prevent dependence on easily spoofed fields. The cleaned traffic is grouped into flows or sessions based on task-specific requirements. A flow is defined as a sequence of packets that share the same 5-tuple: (source IP address, destination IP address, source port, destination port, and the transport layer protocol identifier), $flow^i = \{p^i_1, p^i_2, \dots, p^i_n, i, n \in \mathbb{Z}^+\}$. A session is defined as a pair of flows with both source/destination IP addresses and source/destination port numbers switched to represent a conversation between two endpoints. $session^i = \{flow^i_{(src-dst)}, flow^i_{(dst-src)}, i \in \mathbb{Z}^+\}$. A flow is unidirectional, and a session is bidirectional. After traffic is grouped into sessions, a slicing procedure is performed to divide the sessions into a series of bursts [15] as shown in the following equation(1):

$$burst^i = \begin{cases} burst^i_{(src-dst)} = \{p^i_{(src-dst),1}, \dots, p^i_{(src-dst),k}\} \\ burst^i_{(dst-src)} = \{p^i_{(dst-src),1}, \dots, p^i_{(dst-src),l}\}, \\ i, k, l \in \mathbb{Z}^+, \quad p^i \in session^i, \quad burst^i \subseteq session^i \end{cases} \quad (1)$$

where k and l denote that there are k packets from $flow_{(src-dst)}$, and l packets from $flow_{(dst-src)}$ in this burst. In our model, we try two values for the burst size: $k+l=1000$ or 100. For most comparisons, we use 1000 as the burst size; for some datasets that contain classes with a smaller number of packets, we choose 100 as the burst size for all classes in order to have enough samples per class. So, a burst can have 1000 or 100 packets, and a burst is a subset of a session.

For each packet in a burst, we keep the first 40, 60, or 80 bytes of L2 payload for the next encoding step, since 40 bytes contain IP and transport layer headers, 60 bytes contain extra optional fields and application header, and 80 bytes also contain the first few bytes of the actual payload. NT-Transformer tries all these options and picks one with the best performance. A study of these three L2 byte choices is performed in Section IV-F.

Since encrypted network traffic has no semantic meaning that can be interpreted by humans, in order to leverage the Transformer model, traffic bursts are transformed into word-like tokens similar to natural languages. Specifically, bytes of packets in a burst are encoded as hexadecimal numbers. We use two representations: uni-gram and bi-gram, which mean one byte and two bytes per encoding unit. For the bi-gram representation, the hexadecimal number of every two adjacent bytes is encoded, and for two bytes, the decimal value of each token can range from 0 to $2^{16} - 1 = 65535$, and the encoding dictionary size is 65536 plus the special tokens that are introduced next. For the uni-gram representation, although only $2^8=256$ entries are needed, we still stick with 65536 plus the number of special tokens as our dictionary size to accommodate the packet size and inter-arrival time features that are introduced later. A comparison of these two payload byte representations is conducted in Section IV-G.

Although byte-level feature is the finest granularity of network traffic representation, which represents the nitty-gritty details of a burst, there are still more features that a burst can provide, e.g., if a file is transferred using different applications or services, the bytes of the file are encapsulated in packets, and different applications, services can adopt different packet sizes, and the packet inter-arrival time feature can also reflect the user-specific behaviors, network-specific conditions, and OS-specific patterns [6]. Therefore, we further encode a burst at a coarser level of granularity: flow level. Both packet size and packet inter-arrival time are encoded as hexadecimal numbers as well. Packet size fits well with the same encoding dictionary of size 65536. For packet inter-arrival time, which is too small to fit the encoding dictionary, we multiply 100,000 to each number to scale it while changing the time unit from seconds to hundredths of milliseconds. We use two special tokens, namely byte size separator token [BSS] and size time separator token [STS], to separate byte stream, packet size, and packet inter-arrival time token sequences. [BSS] is used to separate byte stream and packet size, [STS] is used to separate packet size and packet inter-arrival time. In addition, at the beginning of each sequence, a special classification token [CLS] is prepended, and it is used as a sequence-level representation for classification in the final hidden layer. The token sequence is illustrated in equation (2). Unlike the

previous works that require fixed-size input from packets/flows [18], [42] [15], [43], NT-Transformer has the flexibility to adjust input byte sequence length, append or remove the packet size or inter-arrival time sequences to achieve the best performance. At last, labels are attached to the token sequences, which are then ready to be divided into training dataset, validation dataset, and testing dataset according to the ratio of 8:1:1.

$$\begin{aligned} TokenSequence^i = \{[CLS], BT_1^i, BT_2^i, \dots, BT_m^i, [BSS], \\ ST_1^i, ST_2^i, \dots, ST_n^i, [STS], TT_1^i, TT_2^i, \dots, TT_{n-1}^i\} \\ i, m, n \in \mathbb{Z}^+ \\ (BT : ByteToken, ST : SizeToken, TT : TimeToken) \end{aligned} \quad (2)$$

B. Training NT-Transformer

Both the input tokens and their sequence positions are embedded into vectors of dimension $d=768$, the token vectors and position vectors are added together to generate the final embeddings, so the final embedding of each token in the input sequences is represented by the sum of its token embedding and position embedding. The token embeddings contain features of network traffic byte, packet size, and packet inter-arrival time, while the position embeddings help to learn the order and temporal relationships of these features. We use the same sine and cosine functions of different frequencies for positional encodings as introduced in the Transformer paper [8]. The NT-Transformer model is trained on the final labeled embeddings. The core concept of Transformer self-attention is to first calculate the scaled dot product attention of Query (Q), Key (K), and then the scaled attention scores are passed through a softmax function to obtain attention weights that sum to 1, finally, the dot product of attention weights and Value matrix (V) is calculated as shown in equation(3) to get the matrix of attention outputs.

$$\begin{aligned} Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \\ d_k \text{ is the dimension of queries and keys} \end{aligned} \quad (3)$$

Q , K , and V are linearly transformed from the same input matrix X through three learned weight matrices W_Q , W_K , and W_V . The output of the 12 transformer layers is passed to a fully connected layer, and then a softmax function is applied to get the predicted classification probabilities. During the training phase, the model parameters are updated through minimizing the cross-entropy loss function as shown in equation(4), where $\mathbf{y}$ is the true label one-hot encoding vector, and $\hat{\mathbf{y}}$ is the predicted probability vector. Both vectors are of length q . The algorithm is summarized in algorithm 1.

$$l(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{i=1}^q y_i \log \hat{y}_i \quad (4)$$

IV. EXPERIMENTS

In this section, three tasks (Section IV-B) are conducted on six sets of encrypted network traffic data (Section IV-A) to investigate the inefficacy problem of pretrained models on

Algorithm 1 NT-Transformer Algorithm

```

1: procedure NT-TRANSFORMER ALGORITHM
2:   for all  $f \in F$  do ▷ F: pcap/pcapng files
3:      $cleaned\_traffic \leftarrow F$  ▷ traffic cleaning
4:      $sessions \leftarrow cleaned\_traffic$  ▷ group traffic into
       sessions based on 5-tuple
5:      $session\_bursts \leftarrow sessions$  ▷ slice sessions
       into session bursts based on the number of packets
       threshold
6:     for all  $session\_burst \in session\_bursts$  do
7:        $byte\_tokens \leftarrow session\_burst$  ▷ extract
         byte_tokens from session_burst
8:        $size\_tokens \leftarrow session\_burst$  ▷ extract
         size_tokens from session_burst
9:        $time\_tokens \leftarrow session\_burst$  ▷ extract
         time_tokens from session_burst
10:       $burst\_tokens \leftarrow [CLS] + byte\_tokens +$ 
         $[BSS] + size\_tokens + [STS] + time\_tokens +$ 
         $label$  ▷ generate labeled burst tokens
11:       $final\_embeddings \leftarrow token\_embeddings +$ 
         $position\_embeddings \leftarrow burst\_tokens$ 
        ▷ generate token_embeddings and posi-
        position_embeddings, and then final_embeddings
12:       $hidden\_state \leftarrow final\_embeddings$  ▷ get the
        final hidden_state after 12 transformer layers
13:       $O \leftarrow fully - connected\ layer(hidden\_state)$ 
        ▷ get the unnormalized output which is repre-
        sented as a vector of q elements
14:       $\hat{y} \leftarrow softmax(O)$  ▷ get the normalized
        probability distribution vector of size q
15:       $\hat{l} \leftarrow argmax(\hat{y})$  ▷ get the predicted
        classification label
16:    end for
17:  end for
18: end procedure

```

previously unseen datasets, and to prove the effectiveness of our model by comparing it to the state-of-the-art deep learning models. The results are presented in Section IV-D. The Time savings by avoiding pre-training are presented in Section IV-E. Two ablation studies on byte representation using uni-gram and bi-gram and three L2 payload byte choices are performed in Section IV-G and Section IV-F, respectively. A convergence speed analysis is provided in Section IV-H to demonstrate that there is negligible overhead for our model when being applied directly to new datasets without the pre-training step. We further evaluate the models' performance in open-world traffic classification in Section IV-I.

A. Datasets

Table II illustrates the six datasets used in our experiments. The first three are publicly available, while the remaining three are our collections. A subset of special protocols is listed in the fourth column, while the common protocols like TCP, UDP, and HTTP are not included.

1) *Dataset I: ISCX*: The first dataset is the publicly available ISCX VPN-nonVPN traffic dataset [50], which contains

TABLE II
DATASETS

Dataset	Size (GB)	# Flows	Selected protocols	# Labels
ISCX VPN-nonVPN [50]	27.7	220533	TLSv1.2, GQUIC, SFTP, SSDP	20
CSTNET-TLS1.3 [15]	9.1	46372	TLSv1.3	120
USTC-TFC [44]	3.71	9853	TLSv1.2, SSLv3, NBSS, SSDP	20
Streaming(application)	4.5	59103	TLSv1.2, TLSv1, DTLSv1.2	8
File transfer and streaming	5.3	224447	TLSv1.2, SSL	6
Streaming(content)	6.7	142724	TLSv1.2	3

packets from 13 services generated by 16 applications. There are 6 NonVPN services and 7 VPN services, and its size is 27.7 GB. For some of the services, we further divide the data based on applications, so that the number of labels = 20.

2) *Dataset II: CSTNET-TLS1.3*: The second dataset is the publicly available CSTNET-TLS1.3 dataset [15] collected under CSTNET [51], which contains encrypted traffic using the new encryption protocol TLS 1.3. This dataset is 9.1 GB, and has 120 applications, so the number of labels = 120.

3) *Dataset III: USTC-TFC*: The third publicly available dataset is USTC-TFC [44]. This dataset is 3.71 GB and consists of ten types of malware traffic and ten types of normal traffic, so the number of labels = 20.

4) *Dataset IV: Streaming(Application)*: Since the last decade, streaming applications have become more and more popular. Cisco estimated that live video usage accounts for 82% of Internet traffic [52], posing a number of management challenges for the Internet. To facilitate the stream traffic analysis studies, we collected a set of Internet streaming traffic between January and March 2024 as our fourth dataset, which contains flows from eight popular livestreaming applications in China, i.e., Bilibili, Douyin, WeChat, Taobao, JD, Meituan, Weibo, and Zhihu. The size of this dataset is 4.5 GB, and the number of labels = 8.

5) *Dataset V: File Transfer and Streaming*: In the fifth dataset, our goal is to investigate whether network traffic content has patterns and rules that are analogous to phrases, grammar, and idioms in natural languages, which stay relatively invariable over time. We initiated three file transfers of the same file to three different cloud applications: Google Cloud, Tencent Cloud, and Baidu Cloud, in the same hardware setting and network environment. The three sets of file transfer traffic for the first ten minutes were captured. In this dataset, three sets of streaming traffic from YouTube, WeChat Live, and Baidu Live were included for comparison purposes, so we have two services (file transfer and streaming) from three companies: Google, Tencent, and Baidu, respectively. The size of the third dataset is: 5.3 GB, the number of labels = 6, and this dataset was collected in March 2024.

6) *Dataset VI: Streaming(Content)*: The sixth dataset focuses on the analysis of streaming content, which is used to classify streaming traffic based on genres. There are very few studies concerning this problem; most work related to

TABLE III
PREVIOUSLY SEEN DATASETS FOR TRAFFIC CLASSIFICATION MODELS

Models	Dataset I: ISCX	Dataset II: CSTNET-TLS1.3	Dataset III: USTC-TFC	Dataset IV: Streaming (application)	Dataset V: File Transfer and Streaming	Dataset VI: Streaming (content)
FlowPic [24]	✗	✗	✗	✗	✗	✗
GraphDApp [38]	✗	✗	✗	✗	✗	✗
ConViTML [42]	✗	✗	✗	✗	✗	✗
PERT [40]	✗	✗	✗	✗	✗	✗
ET-BERT [15]	✓	✓	✗	✗	✗	✗
TrafficFormer [17]	✓	✗	✗	✗	✗	✗
YaTC [18]	✓	✗	✓	✗	✗	✗
NT-Transformer	✗	✗	✗	✗	✗	✗

this topic is to identify whether a network flow is streaming or not [53], [54], but not the streaming content type [22]. So we collected our sixth dataset containing streaming flows from Bilibili with three genres, i.e., chat, game, and outdoor, in April 2023. The size of this dataset is 6.7 GB, and the number of labels = 3.

B. Tasks

Task 1: Reusability of Pretrained Models on Network Traffic. The first task aims to investigate whether network traffic classification tasks on previously seen and unseen datasets can benefit from pretrained models. Table III summarizes which datasets (out of the six datasets illustrated in Table II) are previously seen for six state-of-the-art traffic classification models. Dataset I (Section IV-A.1) was used to build pretrained models for ET-BERT, TrafficFormer, and YaTC, Dataset II (Section IV-A.2) was used by ET-BERT, and Dataset III (Section IV-A.3) was used by YaTC in their pretrained models, respectively. Dataset IV (Section IV-A.4), Dataset V (Section IV-A.5), and Dataset VI (Section IV-A.6) are previously unseen datasets for all models. Our goal is to verify whether the pretrained models can provide the “once and for many” benefit as intended.

Task 2: Differences between Network Traffic and Natural Languages. The purpose of the second task is to answer the question: “Why do pretrained models provide little benefit when being applied to previously unseen network traffic?” Differences between network traffic and natural languages are studied to validate that there exist unique and intrinsic characteristics in network traffic that differentiate it from natural languages and result in inefficacy of pretrained models. Dataset V: File Transfer and Streaming (Section IV-A.5) is used for this task.

Task 3: Network Traffic Classification Based on Application and Content The third task aims to classify network traffic from two levels of granularity: application and content, to verify the transferability and generalizability of our model. Traffic classification based on application has been widely studied, while very little research has been performed concerning the content classification, because classifying traffic purely based on content from the same service and application without any knowledge of the original videos or any help

from video comments can be challenging [22], [55]. Dataset IV: Streaming(application) (Section IV-A.4) and Dataset VI: Streaming(content) (Section IV-A.6) are used for this task.

C. Implementation and Evaluation Details

Our experiments are implemented with PyTorch 2.0.1, UER [56], and ET-BERT [15]. The jobs are run on NVIDIA GeForce RTX 4090 GPUs. The AdamW optimizer is adopted in the training phase, and the learning rate is set to 2×10^{-5} , the batch size is 32, and the dropout rate is 0.5.

Four evaluation metrics are used: accuracy, precision, recall, and F1 score. Accuracy is calculated as the ratio of correct predictions to the total number of input samples, which tells us how well the model is performing in terms of correct predictions. Precision is calculated as the number of true positive predictions divided by the number of true positive and false positive predictions, which tells us the percentage of positive predictions that are actually positive. Recall is calculated as the number of true positive predictions divided by the actual positive samples, which tells us the percentage of positive samples that are predicted positive. F1 score is defined as the harmonic mean of precision and recall, with a higher value indicating a better balance between precision and recall. In our experimental results, micro accuracy is adopted [57], while macro precision, macro recall, and macro F1 [41] are calculated as follows: the value per label is computed first, and then the average value across all the labels is used as the result presented in the following table. The four evaluation metrics are illustrated in Equations (5) to (8).

$$\text{micro Accuracy} = \frac{\sum_{i=1}^N TP_i}{\text{All predictions}} \quad (5)$$

$$\text{Precision}_i = \frac{TP_i}{TP_i + FP_i}$$

$$\text{macro Precision} = \frac{1}{N} \sum_{i=1}^N \text{Precision}_i \quad (6)$$

$$\text{Recall}_i = \frac{TP_i}{TP_i + FN_i}$$

$$\text{macro Recall} = \frac{1}{N} \sum_{i=1}^N \text{Recall}_i \quad (7)$$

TABLE IV
COMPARISON RESULTS ON ISCX, CSTNET-TLS1.3, AND USTC-TFC DATASETS

Dataset	Dataset I: ISCX				Dataset II: CSTNET-TLS1.3				Dataset III: USTC-TFC			
Model	AC	PR	RC	F1	AC	PR	RC	F1	AC	PR	RC	F1
FlowPic [24]	0.7062	0.7401	0.6995	0.7075	0.3857	0.3783	0.3854	0.3727	—	—	—	—
GraphDApp [38]	0.7608	0.8262	0.7613	0.7436	0.6049	0.6545	0.6055	0.6086	0.8837	0.8818	0.8843	0.8756
ConViTML [42]	0.9647	0.9661	0.9647	0.9648	0.7065	0.7060	0.7065	0.7007	0.9237	0.9308	0.9237	0.9227
PERT [40]	0.9553	0.9560	0.9556	0.9551	0.7551	0.7577	0.7553	0.7549	0.9484	0.9441	0.9412	0.9411
ET-BERT [15]	0.9927	0.9929	0.9928	0.9926	0.7997	0.8069	0.7999	0.8006	0.9645	0.9615	0.9622	0.9594
TrafficFormer [17]	0.9901	0.9902	0.9902	0.9899	0.7727	0.7787	0.7730	0.7721	0.9613	0.9595	0.9579	0.9542
YaTC [18]	0.9912	0.9914	0.9912	0.9912	0.8545	0.8577	0.8545	0.8546	0.9596	0.9620	0.9596	0.9598
NT-Transformer(Byte)	0.9749	0.9758	0.9750	0.9749	0.7344	0.7407	0.7345	0.7341	0.9419	0.9411	0.9397	0.9361
NT-Transformer(Size)	0.7797	0.8009	0.7800	0.7760	0.6866	0.7031	0.6865	0.6872	0.9466	0.9125	0.8956	0.8891
NT-Transformer(Time)	0.8278	0.8294	0.8281	0.8276	0.2873	0.2981	0.2874	0.2893	0.8544	0.8184	0.8032	0.8049
NT-Transformer (Size and Time)	0.7865	0.8166	0.7865	0.7891	0.6918	0.6996	0.6917	0.6919	0.9415	0.9475	0.9437	0.9396
NT-Transformer (All, bi-gram)	0.9809	0.9822	0.9809	0.9808	0.7344	0.7407	0.7345	0.7341	0.9485	0.9444	0.9425	0.9421
NT-Transformer (All, uni-gram)	0.9901	0.9903	0.9902	0.9900	0.8769	0.8801	0.8770	0.8770	0.9593	0.9558	0.9516	0.9521

$$F1_i = \frac{2 \times Precision_i \times Recall_i}{Precision_i + Recall_i}$$

$$macro\ F1 = \frac{1}{N} \sum_{i=1}^N F1_i \quad (8)$$

According to the recently published network traffic classification results [15], [17], [18], [40] Transformer-based pre-training models significantly outperform the state-of-the-art fingerprint-based, feature engineering-based, and traditional deep learning methods on multiple public datasets. Therefore, in our experiments, four Transformer-based pre-training models: PERT, ET-BERT, TrafficFormer, and YaTC are selected for comparisons, two deep learning models: FlowPic, GraphDApp, and one deep learning + Transformer model: ConViTML are also selected to extend the coverage of comparisons. If there is a pretrained model available on GitHub for a Transformer-based model, it is added during the training phase of this model. If a pretrained model is not required, we train the model directly. Each comparison model is trained for different number of epochs to get its best performance, and the input data requirements for different models are met with our best effort, except that some studies only require the MAC addresses to be wiped off from Ethernet headers [42], in our comparisons, all the input data is cleaned and processed as discussed in Section III-A.

D. Comparison Results

1) *Reusability of Pretrained Models on Network Traffic*: The notations (Byte), (Size), (Time), and (Size and Time) appended to NT-Transformer indicate the type of input sequence used: raw byte sequences, packet size sequences, packet inter-arrival time sequences, or a combination of size and time sequences, respectively. NT-Transformer (All) takes the combination of all byte, size and time sequences as potential input tokens and selects the combination with the best performance. Uni-gram means the byte feature is represented using one byte per token, while bi-gram means double adjacent

bytes per token. All the NT-Transformer models do not include a pre-training phase, whereas PERT, ET-BERT, TrafficFormer, and YaTC models do.

As shown in Table IV, our analysis reveals two key trends. First, Transformer-based models generally surpass other approaches. Second, among input features, the byte-level feature proves more informative than packet-level features like packet size or inter-arrival time. This is evidenced by the superior performance of NT-Transformer(Byte) over its variants using only size, time, or even their combined features. Furthermore, a comparison between FlowPic [23] and NT-Transformer(Size and Time) highlights the advantage of preserving raw sequential data. Although both utilize packet size and inter-arrival time, FlowPic transforms this information into images by summing packet counts within fixed time bins of fixed time block sizes (e.g., 60 seconds for FlowPic [23] or 15 seconds for mini-FlowPic [25]). This process discards the fine-grained, packet-by-packet sequence and severs correlations at the imposed time boundary. In contrast, NT-Transformer(Size and Time) processes the original, unaltered sequences, allowing the self-attention mechanism to capture the full context and temporal dependencies, leading to its stronger performance.

On the ISCX dataset, ET-BERT achieves the best performance, followed by YaTC and TrafficFormer, which rank second and third, respectively. Note that the full ISCX dataset is included in the pre-training step of ET-BERT and YaTC, while only the nonVPN part of this data is included in TrafficFormer's pre-training phase, and the downstream classification task is also on the full ISCX dataset; this can explain why TrafficFormer's performance is slightly lower than the other two models. ET-BERT outperforms NT-Transformer(Byte), the latter is ET-BERT without the pre-training model. So the pre-training model pushes the F1 score up by 1.18% when fine-tuning a dataset that has already been included in the pre-training step. NT-Transformer(All, bi-gram) increases the F1 score by 0.59% compared with

TABLE V
COMPARISON RESULTS ON STREAMING(application), FILE TRANSFER AND STREAMING, AND STREAMING(content) DATASETS

Dataset	Dataset IV: Streaming(application)				Dataset V: File Transfer and Streaming				Dataset VI: Streaming(content)			
Model	AC	PR	RC	F1	AC	PR	RC	F1	AC	PR	RC	F1
FlowPic [24]	0.6960	0.6944	0.7358	0.6922	0.9207	0.8943	0.9242	0.9169	0.8333	0.8410	0.8329	0.8323
GraphDApp [38]	0.9160	0.9222	0.9162	0.9166	0.9808	0.9810	0.9808	0.9807	0.6335	0.6347	0.6338	0.6330
ConViTML [42]	0.9781	0.9787	0.9781	0.9782	0.9574	0.9615	0.9574	0.9573	0.9563	0.9564	0.9563	0.9563
PERT [40]	0.9811	0.9812	0.9811	0.9811	0.9659	0.9662	0.9660	0.9658	0.9742	0.9747	0.9741	0.9740
ET-BERT [15]	0.9845	0.9843	0.9843	0.9843	0.9770	0.9798	0.9771	0.9771	0.9744	0.9750	0.9743	0.9746
TrafficFormer [17]	0.9829	0.9834	0.9828	0.9829	0.9847	0.9848	0.9848	0.9848	0.9789	0.9790	0.9787	0.9790
YaTC [18]	0.9844	0.9846	0.9844	0.9844	0.9885	0.9887	0.9885	0.9885	0.9773	0.9778	0.9773	0.9774
NT-Transformer(Byte)	0.9814	0.9814	0.9814	0.9814	0.9847	0.9853	0.9848	0.9847	0.9744	0.9750	0.9743	0.9743
NT-Transformer(Size)	0.9798	0.9799	0.9798	0.9798	0.9732	0.9733	0.9735	0.9732	0.7074	0.7037	0.7077	0.7043
NT-Transformer(Time)	0.7003	0.7061	0.7001	0.6905	0.8927	0.8938	0.8930	0.8925	0.8250	0.8313	0.8250	0.8260
NT-Transformer (Size and Time)	0.9815	0.9821	0.9814	0.9816	0.9611	0.9633	0.9612	0.9610	0.7133	0.7153	0.7133	0.7140
NT-Transformer (All, bi-gram)	0.9860	0.9861	0.9858	0.9861	0.9866	0.9868	0.9868	0.9865	0.9804	0.9810	0.9803	0.9803
NT-Transformer (All, uni-gram)	0.9867	0.9875	0.9866	0.9869	0.9943	0.9945	0.9943	0.9942	0.9864	0.9863	0.9863	0.9860

NT-Transformer(Byte) because the additional flow level features are appended to the input tokens. A similar pattern is also evident in the results on the USTC-TFC dataset. YaTC yields the best performance in terms of F1 score, ET-BERT, and TrafficFormer rank second and third, since this dataset is included in the pre-training phase of YaTC, the F1 scores of ET-BERT, TrafficFormer, and NT-Transformer are slightly lower than YaTC, which indicates that for this dataset, pretrained models gives the performance some but limited boost as well.

However, pretrained models lose their advantages on the CSTNET-TLS1.3 dataset, NT-Transformer with uni-gram representation outperforms the other models with more than 2.24% increase on F1 score. Specifically, NT-Transformer(uni-gram) increases the F1 score by 7.64% compared to ET-BERT, which has this dataset included in its pre-training phase. For this experiment, NT-Transformer(uni-gram) picks only the first 40 bytes of each packet instead of 60 or 80 bytes, which means the application header and the first few bytes of payload fail to provide any useful information for traffic classification because this dataset is fully encrypted. In models with the pre-training phase, fixed bytes and byte representations are used to build the pretrained models, e.g., ET-BERT and TrafficFormer keep the first 64 bytes of each packet, YaTC picks up to the first 320 bytes, and both ET-BERT and TrafficFormer adopt the bi-gram representation. These fixed patterns can affect the performance of pretrained models when dealing with complicated traffic classification jobs, such as up to 120 labels and encrypted payload.

Pretrained models fail to provide any benefit when applied to the three previously unseen datasets, which contain streaming and file transfer traffic. These datasets themselves were not included in the pre-training step of any models; however, streaming and file transfer traffic from various applications were included, which means the patterns and rules of these traffic types have already been learned by the pretrained models.

From the F1 scores presented in Table V, we can observe that NT-Transformer(All, uni-gram) outperforms all the models that include the pre-training step, specifically, the F1 score of NT-Transformer(All, uni-gram) is 0.25%, 0.57%, and 0.7%, higher than other models, respectively, which indicates the pretrained model is not as effective as intended when being applied to new datasets that are not included in the pre-training step, although the service types of the new datasets' traffic have been included in the pretrained model. This implies that besides the service type, there are some particular patterns or rules specific to the new dataset that are not learned by the pretrained models, and the learned application-level or payload-level features may not add value to downstream tasks with new datasets as input. The intention of the pretrained model is to improve the performance and speed up the process of specific downstream tasks, since the pre-training process is usually expensive and time-consuming, and based on the results discussed above, this intention is not met.

2) *Differences Between Network Traffic and Natural Languages*: Based on the findings from the first task discussed above, we further investigate the differences between network traffic and natural languages in this second task. We transfer the same file to three different cloud applications, with three streaming flows added for comparison purposes, as discussed in Section IV-A.5.

From the results shown in Table V, NT-Transformer(All, uni-gram) outperforms all the other models. Although NT-Transformer(Size) and NT-Transformer(Time) individually underform NT-Transformer(Byte), they both can give NT-Transformer(Byte) a boost when size and time features are appended to the byte tokens in this experiment. This particular file being transferred was not used in the pre-training step; however, extensive file transfer data of diverse services was included. This indicates there are differences between NLP and network traffic analysis: for NLP, pretrained models learn from a big corpus of language, the phrases, grammar, slang, etc., of which will be conformed or reused in future communications.

In this experiment, transferring the same file to three different cloud applications is analogous to reading the same paragraph to three people individually. In the latter case, the three people would interpret the same information from the same paragraph. In network traffic analysis, NT-Transformer(All, uni-gram) is able to identify the difference of these three transfers and classify the transfer traffic into three different categories, which means different applications and cloud environments add their unique signatures to the data being transferred using them. Every flow is unique, even though the actual user data is the same. The accurate identification of added streaming traffic from applications that are products of the same companies as the file transfer applications shows that different services (e.g., file transfer and streaming) have their own special characteristics. These results prove our hypothesis in Section III is true.

3) *Network Traffic Classification Based on Application and Content*: In the third task, besides classifying network traffic based on application or service, we also focus on the streaming content classification. As shown in Table V, NT-Transformer(uni-gram) achieves at least a 0.25% improvement compared to all the other pretrained models on Dataset IV, and a 0.7% improvement on Dataset VI. For Dataset IV, the first 80 bytes plus packet size sequences are used, and for Dataset VI, the first 40 bytes sequences yield the best performance. These experiments prove that NT-Transformer has strong generalization ability; it can learn task-specific network traffic features during the training step for diverse types of downstream tasks. The pretrained model will help when the input data to downstream tasks is a subset of the data used for pre-training; however, under the scenarios of newly-collected datasets, even though with previously learned service types, pretrained models provide little benefit, which may put the value of applying pretrained models to the network traffic classification field into doubt.

Unlike phrases, grammar, and idioms in natural languages that can be learned once and used many times for language-related downstream tasks, network traffic features contain data-specific patterns, user-specific behaviors, network-specific conditions, and application-specific signatures. These features are ever-changing and unpredictable, and the combination of all these features and encryption techniques undermines the reusability of pretrained models.

Therefore, we should not expect a pretrained network traffic model that only needs to be trained once on a tremendous amount of data, and then it can facilitate our related future network traffic analysis tasks on any newly-collected dataset, even if the newly-collected dataset is of previously learned service types. Also, learned traffic features like application headers and payload data may affect the performance of pretrained models, since these fields are encrypted, which does not convey any useful information in future downstream tasks.

Instead, a network classification model like NT-Transformer that does not require any pretrained model, and takes both packet-level and flow-level features that can capture task-specific network traffic features based on the user's specific requirements, is recommended for network traffic analysis tasks. Because NT-Transformer can capture the details of

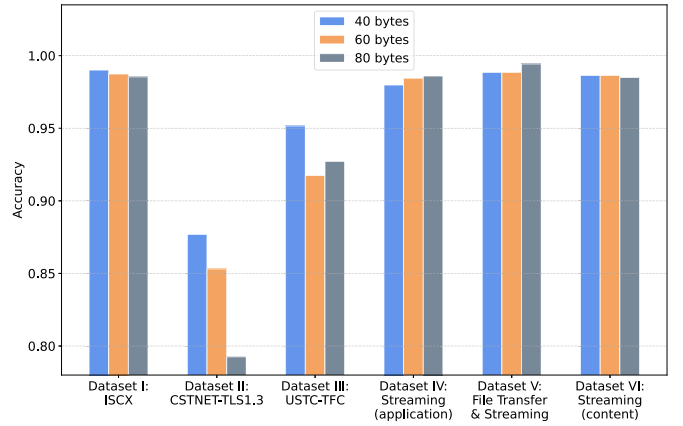


Fig. 3. Ablation study on sample size of L2 payload (40 bytes, 60 bytes and 80 bytes).

data, application, user, network, etc., that can influence the network traffic we are interested in. The ability to successfully classify network traffic based on traffic content demonstrates the transferability and generalizability of NT-Transformer.

E. Analysis of the Time Savings Achieved by Avoiding Pre-Training

It is well-known that the pre-training step is highly time-consuming, with the required duration depending on the model architecture, GPU hardware, and the volume of input data. A key advantage of NT-Transformer is the substantial time savings it achieves by eliminating this step. To quantify this, we conducted an analysis using the implementation described in Section IV-C, comparing our approach against two pre-training-based baselines: ET-BERT and TrafficFormer.

We followed their prescribed pre-training procedure using the Masked Burst Model (MBM) task. In this task, each token in the input sequence is randomly masked with a 15% probability. The chosen token is then replaced with a [MASK] token 80% of the time, replaced with a random token 10% of the time, or left unchanged 10% of the time. Our experiments showed that pre-training on 1 GB and 2 GB of input data took 2.65 hours and 5.24 hours, respectively, indicating a near-linear relationship between pre-training time and data size. Given that the full pre-training datasets for ET-BERT and TrafficFormer are approximately 30 GB and 18.8 GB, the estimated total pre-training time for the MBM task alone is about 79.5 hours and 49.82 hours, respectively. It is important to note that both models require additional pre-training tasks—ET-BERT uses the Same-origin BURST Prediction (SBP) task, and TrafficFormer uses the Same Origin-Direction-Flow (SODF) task—which would further increase the total pre-training time.

The time savings achieved by forgoing pre-training are tremendous, especially when contrasted with the much shorter fine-tuning phase. Pre-training occurs on the order of days, while fine-tuning typically requires only minutes/hours. This stark difference highlights a major efficiency gain offered by the NT-Transformer approach.

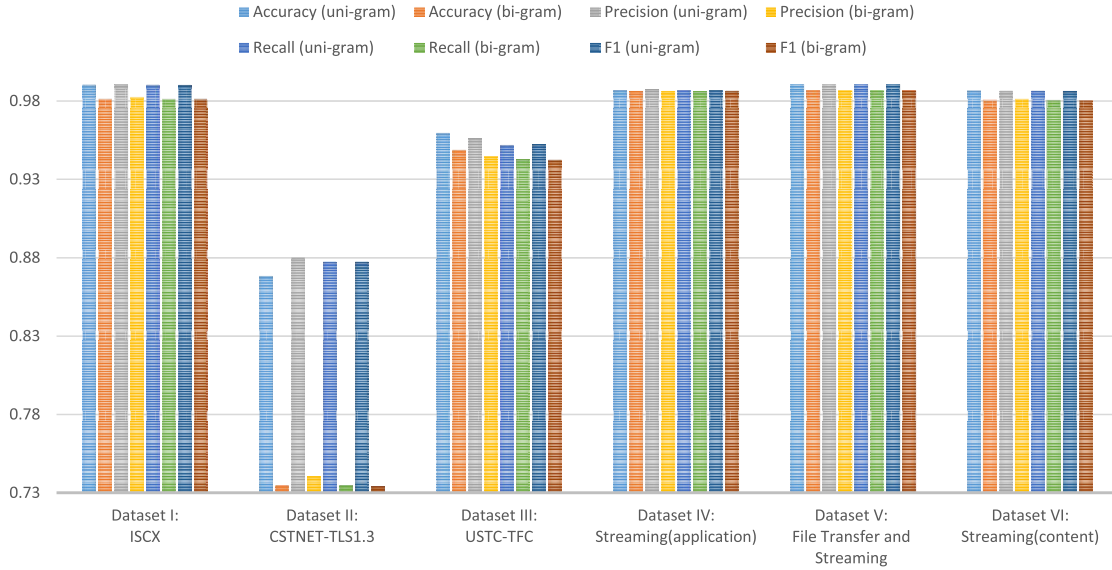


Fig. 4. Ablation study on byte token representation (uni-gram vs. bi-gram).

F. Ablation Study on Sample Size of L2 Payload (40 Bytes, 60 Bytes, and 80 Bytes)

As mentioned in Section III-A, NT-Transformer tries the first 40, 60, and 80 bytes of L2 payload for encoding and picks the one that yields the best performance. In this section, we discuss the performance difference of these three options.

As illustrated in Figure 3, the first 40 bytes achieves the highest accuracy on four datasets: ISCX, CSTNET-TLS1.3, USTC-TFC, and Streaming(content). This indicates that the application header and application payload fail to provide effective information on traffic classes due to the encryption techniques applied to them, especially for the CSTNET-TLS1.3 dataset, which is fully encrypted using TLS 1.3. For the ISCX and USTC-TFC datasets, there exists a small percentage of flows that are unencrypted, e.g., TCP, UDP, HTTP, and FTP traffic; therefore, the application header and actual data bytes can convey some information on traffic classes for the unencrypted flows, so the performance difference among the three byte choices is smaller. The other two datasets, Streaming (application) and File Transfer and Streaming, contain a higher proportion of unencrypted traffic, which explains why 80 bytes slightly outperforms 40 bytes on these two datasets.

This study proves our flexible choice of L2 bytes is advantageous compared to fixed input, because different datasets may consist of different amounts of encrypted traffic for which only IP and transport headers are beneficial in traffic classification. The application header and user data can be considered as “garbled text” since they are encrypted.

G. Ablation Study on Byte Token Representation (Uni-Gram Vs. Bi-Gram)

The related works mainly adopt the bi-gram representation to represent the network traffic byte token pool, analogous to the common word pool of a natural language. Due to the unique characteristics of network traffic, the bi-gram

representation may not be optimal; therefore, an ablation study is conducted to compare the byte representation using uni-gram and bi-gram. For this study, we use the six datasets introduced in Section IV-A.

From the results shown in Figure 4, we can observe that for all six datasets, the uni-gram representation outperforms the bi-gram representation, with 0.92%, 14.29%, 1%, 0.08%, 0.77%, and 0.57% improvement of F1 score, respectively. This may seem counterintuitive, since bi-gram has 65536 encoding possibilities, while uni-gram only has 256. However, a bi-gram is two uni-grams plus relative bit position information, and in NT-Transformer(All, uni-gram), with position embeddings added to token embeddings, the temporal relations among all the byte tokens are learned, so no byte or position information is lost in the uni-gram representation, and what’s more, for the same amount of dataset, uni-gram representation can actually provide richer input features, i.e., the occurrences of the 256 possibilities of byte tokens and their temporal positions, since there is a higher probability for the same uni-gram token to reappear for the model to learn, while for bi-gram with 65536 possibilities, this reappearance probability is much lower. Therefore, in order to learn the relations of byte tokens and temporal positions, more input data is needed for the bi-gram representation.

H. Convergence Study of the Training Step (With Pretrained Model Vs. Without Pretrained Model)

Since accuracy and speed are two benefits that pretrained models are supposed to bring to the downstream tasks, accuracy is discussed in the above sections. In this section, the convergence speed of the training phase is studied using the CSTNET-TLS1.3 dataset. Experiments on the other datasets exhibit similar patterns.

From Figure 5, we can tell that the pretrained model gives the downstream task a good start: after the first epoch, the highest accuracy is 0.4624, which is achieved by ET-BERT,

TABLE VI
CLASS VS. ALL OPEN-WORLD COMPARISONS

Dataset	Streaming vs. All				File transfer vs. All			
Model	AC	PR	RC	F1	AC	PR	RC	F1
FlowPic [24]	0.6384	0.6505	0.6385	0.6293	0.7017	0.7042	0.7020	0.7012
GraphDApp [38]	0.8500	0.8702	0.8500	0.8479	0.5167	0.7542	0.5167	0.3693
ConViTML [42]	0.6558	0.6732	0.6565	0.6545	0.5000	0.5000	0.5000	0.3608
PERT [40]	0.8364	0.8456	0.8370	0.8356	0.6167	0.6340	0.6175	0.6155
ET-BERT [15]	0.8667	0.8820	0.8670	0.8655	0.6333	0.6588	0.6333	0.6303
TrafficFormer [17]	0.8333	0.8473	0.8343	0.8323	0.6500	0.6625	0.6513	0.6495
YaTC [18]	0.8444	0.8662	0.8438	0.8425	0.7019	0.7225	0.6959	0.6859
NT-Transformer	0.8833	0.8945	0.8835	0.8825	0.6717	0.6920	0.6722	0.6715
NT-Transformer(Size and Time)	0.6833	0.7700	0.6835	0.6555	0.6020	0.6119	0.6089	0.6015

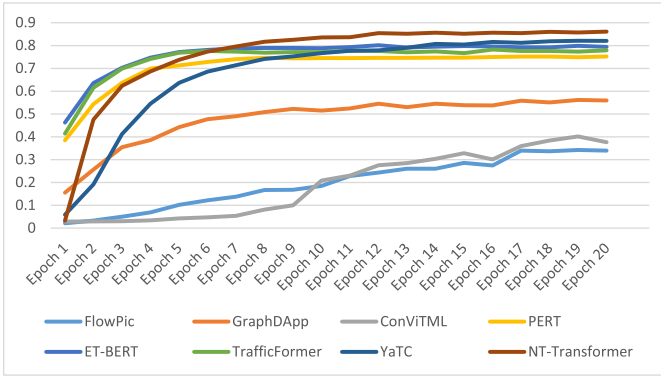


Fig. 5. Convergence study of the training step on the CSTNET-TLS1.3 Dataset.

because the CSTNET-TLS1.3 dataset is included in the pre-training step of ET-BERT. However, the growing speed of accuracy is faster of NT-Transformer, which gradually catches up and overtakes ET-BERT within 6 epochs to reach the highest training accuracy of 0.7736. After this point, NT-Transformer's training accuracy keeps growing until around 0.88, and there is no advantage in terms of training accuracy that can be observed for the tasks with pretrained models.

I. Open-World Comparisons of the Class Vs. All Problems and Future Work

Moving beyond traditional closed-world evaluations, we investigate open-world network traffic classification in this section. Inspired by FlowPic's class vs. all approach [24], we conducted experiments at two levels of granularity.

First, in a coarse-grained class vs. all setting, models were trained to distinguish a target service (e.g., streaming) from all others. During testing, the target class was replaced with an unseen application (e.g., training on YouTube, WeChat Live, etc., testing on Zhihu Live). Results in Table VI show that while this is an easier task than fine-grained service+application classification (discussed in Section IV-D), model performance drops significantly when confronted with unseen traffic. NT-Transformer excelled in streaming classification, potentially due to its lack of a pre-training phase, while FlowPic's strength in capturing flow burstiness made it best for file transfer classification.

Second, in a more challenging fine-grained setting, we trained models on ten specific service+application categories (e.g., Bilibili Live, Douyin, Google File Transfer) and replaced three with unseen counterparts from the same coarse service (e.g., testing WeChat Live for Bilibili Live, JD Live for Douyin, Tencent Cloud for Google Cloud file transfer). In this scenario, all models failed. For instance, NT-Transformer, having learned features of a specific service+application, could not generalize to a different one within the same service.

These findings highlight the significant challenge of open-world classification, especially at finer granularities. The goal is a model that captures generalizable features common to different services across diverse applications, while avoiding overfitting to the specifics of individual service-application flows. In other words, the model needs to learn features that are discriminative enough to distinguish between traffic categories, yet sufficiently abstract to generalize beyond any single instance. Significant progress on this issue has yet to be made by the research community, positioning it as a priority for our future work.

V. CONCLUSION

In this paper, the reusability and transferability of pretrained models in the applications of the network traffic classification field are studied in detail. A Transformer-based encrypted traffic classification model called NT-Transformer is proposed. NT-Transformer(All, uni-gram) can learn traffic features from both byte level and flow level, without the relatively expensive pre-training procedure using unlabeled data. Experiment results show that the uni-gram representation of traffic bytes achieves the optimal performance, and NT-Transformer can push the F1 score up by at least 0.25% - 2.24% on one public dataset and three newly collected datasets. Extending from this work, we will explore more advanced approaches to tackle the open-world traffic classification problem.

REFERENCES

- [1] (2014). *Google Transparency Report*. [Online]. Available: <https://transparencyreport.google.com/https/overview?hl=en>
- [2] (2024). *Year in Review 2024*. [Online]. Available: <https://radar.cloudflare.com/year-in-review/2024>
- [3] (2024). *Visualizing 2023 Web Traffic Trends Through ATI's Lens*. [Online]. Available: <https://www.keysight.com/blogs/en/tech/nwvs/2024/01/24/ati-2023-traffic-analysis>

- [4] (2024). *Global Internet Usage Trends in 2024*. [Online]. Available: <https://hypestat.com/blog/global-internet-usage-trends-in-2024>
- [5] Z. Zou, J. Ge, H. Zheng, Y. Wu, C. Han, and Z. Yao, "Encrypted traffic classification with a convolutional long short-term memory neural network," in *Proc. IEEE 20th Int. Conf. High Perform. Comput. Commun., IEEE 16th Int. Conf. Smart City, IEEE 4th Int. Conf. Data Sci. Syst. (HPCC/SmartCity/DSS)*, Jun. 2018, pp. 329–334.
- [6] S. Rezaei and X. Liu, "Deep learning for encrypted traffic classification: An overview," *IEEE Commun. Mag.*, vol. 57, no. 5, pp. 76–81, May 2019.
- [7] W. Wang, M. Zhu, J. Wang, X. Zeng, and Z. Yang, "End-to-end encrypted traffic classification with one-dimensional convolution neural networks," in *Proc. IEEE Int. Conf. Intell. Secur. Informat. (ISI)*, Jul. 2017, pp. 43–48.
- [8] A. Vaswani et al., "Attention is all you need," in *Proc. NIPS*, 2017, pp. 6000–6010.
- [9] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Hum. Lang. Technol.*, vol. 1, Jun. 2019, pp. 4171–4186.
- [10] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut, "ALBERT: A Lite BERT for Self-supervised Learning of Language Representations," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2020.
- [11] A. Dosovitskiy et al., "An image is worth 16×16 words: Transformers for image recognition at scale," in *Proc. 9th Int. Conf. Learn. Represent.*, 2021.
- [12] A. Gulati et al., "Conformer: Convolution-augmented transformer for speech recognition," in *Proc. Interspeech*, Oct. 2020, pp. 5036–5040.
- [13] A. Shin, M. Ishii, and T. Narihira, "Perspectives and prospects on transformer architecture for cross-modal tasks with language and vision," *Int. J. Comput. Vis.*, vol. 130, no. 2, pp. 435–454, Feb. 2022.
- [14] S. Islam et al., "A comprehensive survey on applications of transformers for deep learning tasks," *Expert Syst. Appl.*, vol. 241, May 2024, Art. no. 122666.
- [15] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proc. ACM Web Conf.*, New York, NY, USA, Apr. 2022, pp. 633–642.
- [16] Q. Pan, Y. Yu, H. Yan, M. Wang, and B. Qi, "FlowBERT: An encrypted traffic classification model based on transformers using flow sequence," in *Proc. IEEE 22nd Int. Conf. Trust, Secur. Privacy Comput. Commun. (TrustCom)*, Nov. 2023, pp. 133–140.
- [17] G. Zhou, X. Guo, Z. Liu, T. Li, Q. Li, and K. Xu, "TrafficFormer: An efficient pre-trained model for traffic data," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2025, pp. 1844–1860.
- [18] R. Zhao et al., "Yet another traffic classifier: A masked autoencoder based traffic transformer with multi-level flow representation," in *Proc. AAAI Conf. Artif. Intell.*, 2023, vol. 37, no. 4, pp. 5420–5427.
- [19] T. van Ede et al., "FlowPrint: Semi-supervised mobile-app fingerprinting on encrypted network traffic," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, Feb. 2020.
- [20] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, "Robust smartphone app identification via encrypted network traffic analysis," *IEEE Trans. Inf. Forensics Security*, vol. 13, no. 1, pp. 63–78, Jan. 2018.
- [21] K. Al-Naami et al., "Adaptive encrypted traffic fingerprinting with bi-directional dependence," in *Proc. 32nd Annu. Conf. Comput. Secur. Appl.*, Dec. 2016, pp. 177–188.
- [22] Y. Xie, H. Deng, L. Peng, and Z. Chen, "Accurate identification of internet video traffic using byte code distribution features," in *Proc. Int. Conf. Algorithms Archit. Parallel Process.*, 2018, pp. 46–58.
- [23] T. Shapira and Y. Shavitt, "FlowPic: Encrypted internet traffic classification is as easy as image recognition," in *Proc. IEEE INFOCOM - IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, Apr. 2019, pp. 680–687.
- [24] T. Shapira and Y. Shavitt, "FlowPic: A generic representation for encrypted traffic classification and applications identification," *IEEE Trans. Netw. Service Manage.*, vol. 18, no. 2, pp. 1218–1232, Jun. 2021.
- [25] E. Horowicz, T. Shapira, and Y. Shavitt, "A few shots traffic classification with mini-FlowPic augmentations," in *Proc. 22nd ACM Internet Meas. Conf.*, 2022, pp. 647–654.
- [26] N. Malekghaini et al., "AutoML4ETC: Automated neural architecture search for real-world encrypted traffic classification," *IEEE Trans. Netw. Service Manage.*, vol. 21, no. 3, pp. 2715–2730, Jun. 2024.
- [27] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep fingerprinting: Undermining website fingerprinting defenses with deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Toronto, ON, Canada, Oct. 2018, pp. 1928–1943.
- [28] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "FS-Net: A flow sequence network for encrypted traffic classification," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, Apr. 2019, pp. 1171–1179.
- [29] C. Liu, Z. Cao, G. Xiong, G. Gou, S.-M. Yiu, and L. He, "MaMPF: Encrypted traffic classification based on multi-attribute Markov probability fingerprints," in *Proc. IEEE/ACM 26th Int. Symp. Quality Service (IWQoS)*, Jun. 2018, pp. 1–10.
- [30] M. Lopez-Martin, B. Carro, A. Sanchez-Esguevillas, and J. Lloret, "Network traffic classifier with convolutional and recurrent neural networks for Internet of Things," *IEEE Access*, vol. 5, pp. 18042–18050, 2017.
- [31] X. Wang, S. Chen, and J. Su, "App-net: A hybrid neural network for encrypted mobile traffic classification," in *Proc. IEEE INFOCOM - IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, Jul. 2020, pp. 424–429.
- [32] K. Lin, X. Xu, and H. Gao, "TSCRNN: A novel classification scheme of encrypted traffic based on flow spatiotemporal features for efficient management of IIoT," *Comput. Netw.*, vol. 190, May 2021, Art. no. 107974.
- [33] A. Nascita, A. Montieri, G. Aceto, D. Ciunzo, V. Persico, and A. Pescapé, "XAI meets mobile traffic classification: Understanding and improving multimodal deep learning architectures," *IEEE Trans. Netw. Service Manage.*, vol. 18, no. 4, pp. 4225–4246, Dec. 2021.
- [34] G. Aceto, D. Ciunzo, A. Montieri, V. Persico, and A. Pescapé, "MIRAGE: Mobile-app traffic capture and ground-truth creation," *Tech. Rep.*, 2019, doi: [10.21227/maj9-vh13](https://doi.org/10.21227/maj9-vh13).
- [35] G. Aceto, D. Ciunzo, A. Montieri, and A. Pescapé, "DISTILLER: Encrypted traffic classification via multimodal multitask deep learning," *J. Netw. Comput. Appl.*, vols. 183–184, Jun. 2021, Art. no. 102985.
- [36] A. Nascita, A. Montieri, G. Aceto, D. Ciunzo, V. Persico, and A. Pescapé, "Improving performance, reliability, and feasibility in multimodal multitask traffic classification with XAI," *IEEE Trans. Netw. Service Manage.*, vol. 20, no. 2, pp. 1267–1289, Jun. 2023.
- [37] I. Guarino, G. Aceto, D. Ciunzo, A. Montieri, V. Persico, and A. Pescapé, "Contextual counters and multimodal deep learning for activity-level traffic classification of mobile communication apps during COVID-19 pandemic," *Comput. Netw.*, vol. 219, Dec. 2022, Art. no. 109452.
- [38] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, "Accurate decentralized application identification via encrypted traffic analysis using graph neural networks," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2367–2380, 2021.
- [39] T.-L. Huoh, Y. Luo, P. Li, and T. Zhang, "Flow-based encrypted network traffic classification with graph neural networks," *IEEE Trans. Netw. Service Manage.*, vol. 20, no. 2, pp. 1224–1237, Jun. 2023.
- [40] H. Y. He, Z. Guo Yang, and X. N. Chen, "PERT: Payload encoding representation from transformer for encrypted traffic classification," in *Proc. ITU Kaleidoscope, Ind.-Driven Digit. Transformation (ITU K)*, Dec. 2020, pp. 1–8.
- [41] P. Lin, K. Ye, Y. Hu, Y. Lin, and C.-Z. Xu, "A novel multimodal deep learning framework for encrypted traffic classification," *IEEE/ACM Trans. Netw.*, vol. 31, no. 3, pp. 1369–1384, Jun. 2023.
- [42] L. Yang, S. Guo, D. Liu, Y. Zeng, X. Jiao, and Y. Zhou, "ConViTML: A convolutional vision transformer-based meta-learning framework for real-time edge network traffic classification," *IEEE Trans. Netw. Service Manage.*, vol. 21, no. 3, pp. 3344–3357, Jun. 2024.
- [43] J. Dai, X. Xu, and F. Xiao, "GLADS: A global-local attention data selection model for multimodal multitask encrypted traffic classification of IoT," *Comput. Netw.*, vol. 225, Apr. 2023, Art. no. 109652.
- [44] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Jan. 2017, pp. 712–717.
- [45] A. W. Moore, D. Zuev, and M. Crogan, "Discriminators for use in flow-based classification," Queen Mary Univ. London, London, U.K., Tech. Rep., Aug. 2005.
- [46] W. Wang et al., "HAST-IDS: Learning hierarchical spatial-temporal features using deep neural networks to improve intrusion detection," *IEEE Access*, vol. 6, pp. 1792–1806, 2018.
- [47] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?," in *Proc. 7th Int. Conf. Learn. Represent. (ICLR)*, 2019.
- [48] T. Lin, Y. Wang, X. Liu, and X. Qiu, "A survey of transformers," *AI Open*, vol. 3, pp. 111–132, Oct. 2022.
- [49] J. Kaplan et al., "Scaling laws for neural language models," 2020, *arXiv:2001.08361*.

- [50] G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of encrypted and VPN traffic using time-related features," in *Proc. 2nd Int. Conf. Inf. Syst. Secur. Privacy*, 2016, pp. 407–414.
- [51] *CSTNET*. Accessed: May 18, 2025. [Online]. Available: <https://www.cstnet.cn/>
- [52] *Visual Network Index (VNI) Complete Prediction*. Accessed: Jul. 12, 2024. [Online]. Available: https://www.cisco.com/c/zh_cn/solutions/service-provider/visual-networking-index-vni/index.html
- [53] Y.-N. Dong, J.-J. Zhao, and J. Jin, "Novel feature selection and classification of internet video traffic based on a hierarchical scheme," *Comput. Netw.*, vol. 119, pp. 102–111, Jun. 2017.
- [54] X. Mu and W. Wu, "A parallelized network traffic classification based on hidden Markov model," in *Proc. Int. Conf. Cyber-Enabled Distrib. Comput. Knowl. Discovery*, Oct. 2011, pp. 107–112.
- [55] J. Choi and X. Zhang, "Classifications of restricted web streaming contents based on convolutional neural network and long short-term memory (CNN-LSTM)," *J. Internet Services Inf. Secur.*, vol. 12, no. 3, pp. 49–62, 2022.
- [56] Z. Zhao et al., "UER: An open-source toolkit for pre-training models," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2019, pp. 241–246.
- [57] N. S. Suhaimi, Z. Othman, and M. R. Yaakub, "Comparative analysis between macro and micro-accuracy in imbalance dataset for movie review classification," in *Proc. 7th Int. Congr. Inf. Commun. Technol.*, 2022, pp. 83–93.



Peiqiang Liu received the Ph.D. degree from Shandong University, China, in 2013. He is currently a Professor at the School of Computer Science and Technology, Shandong Technology and Business University, China. His research interests include algorithms and complexity theory and computational biology.



Yingshuo Bao received the B.S. degree in computer science and technology from Minnan Normal University, Zhangzhou, China, in 2020, and the M.S. degree in computer science and technology from the University of Jinan, Jinan, China, in 2023. He is currently pursuing the Ph.D. degree in systems analysis and integration with the Department of Systems Science, Faculty of Arts and Sciences, Beijing Normal University, Zhuhai, China. His research interests include network traffic analysis, causal inference, graph neural networks, human physiological information monitoring.



Zhenzhen Yan (Member, IEEE) received the Ph.D. degree from the University of Virginia in 2013. She is currently an Assistant Professor at the School of Computer Science and Technology, Shandong Technology and Business University, China. Her research interests include computer networks, traffic engineering, and network security.



Lizhi Peng (Member, IEEE) received the B.Eng. degree from Xi'an Jiaotong University in 1998, the M.Eng. degree from the University of Jinan, Jinan, China, in 2006, and the Ph.D. degree from Harbin Institute of Technology in 2015. He was a Visitor Scholar with the Department of Computer Science, University of Otago, New Zealand, in 2017. He is currently a Professor with the School of Information Science and Engineering, University of Jinan. His current research interests include machine learning, evolutionary computing, computer networks, and parallel computing.



Bo Yang (Member, IEEE) received the M.Sc. degree from Southeast University, Nanjing, China, in 1989, and the Ph.D. degree in computational materials science from Wuhan University of Technology, Wuhan, China, in 2006. He is currently a Professor with the University of Jinan, Jinan, China, where he is the Director of Shandong Provincial Key Laboratory of Network-Based Intelligent Computing. He is also the Deputy Director of the Quan Cheng Laboratory, Jinan. His main research interests include intelligent computing, computer networks, machine learning, knowledge discovery, and intelligent control. He was a member of numerous program committees of international conferences in the area of intelligent computing. He is the Chair of Shandong Computer Federation and the Vice-Chair of the ACM SigCSE China. He is the Chief Editor for *Information Technology and Informatization*. He is on the Editorial Board of *Journal of Communications*.